La Asociación de Antiguos Alumnos de un centro universitario edita una revista anual en la que informa a sus asociados sobre todas las actividades desarrolladas por la Asociación, además de ofrecer artículos de interés para sus asociados. A partir de los datos de todos los números publicados hasta la fecha de la revista, se pretende obtener unas estadísticas sobre sus contenidos y su evolución. Para ello, se dispone de la siguiente información sobre cada número de la revista:

- **Número:** número de orden de la revista.
- **Fecha:** fecha de publicación de la revista.
- **Número de ejemplares:** número total de ejemplares impresos de la revista.
- **Número de páginas fijas:** número de páginas que ocupan las secciones fijas de la revista, como la portada, la editorial o las noticias.
- **Número de anuncios:** número de anuncios que contiene la revista.
- **Artículos:** lista de artículos publicados en la revista. Para cada artículo se tiene su título, su categoría y su número de páginas.

Una línea ejemplo del archivo csv en el que están registrados los datos es la siguiente:

```
286;24/03/2025;760;20;3;[Web accesible - desarrollo web - 8, Sesgo en algoritmos - ética profesional - 12]
```

Esta línea indica que el número 286 de la revista se publicó el 24 de marzo de 2025. De este número se imprimieron 760 ejemplares. El número de páginas fijas es 20 y el número de anuncios es 3. La revista contiene 2 artículos: el artículo 'Web accesible' es de la categoría 'desarrollo web' y tiene 8 páginas; el artículo 'Sesgo en algoritmos' es de la categoría 'ética profesional' y tiene 12 páginas.

Para analizar los datos, **implemente en Java** lo especificado en los siguientes ejercicios:

Ejercicio 1: Tipo Articulo (0,5 ptos)

Implemente en el paquete `fp.asociacion` el tipo Articulo mediante un **record,** de acuerdo con la siguiente descripción:

Propiedades:
- **Título**, de tipo String, consultable.
- **Categoria**, de tipo String, consultable.
- **Páginas**, de tipo Integer, consultable.
- **Articulo largo**, de tipo Boolean, consultable. Un artículo se considera largo si tiene 10 o más páginas. Implemente el valor 10 como una constante.

Constructores:
- C1: recibe un parámetro por cada propiedad básica del tipo, en el mismo orden en el que están definidas.

Representación como cadena: una cadena con todas las propiedades básicas del tipo.
Restricciones:
- R1: El título no puede estar vacío.
- R2: El número de páginas debe ser mayor que 0.

Criterio de igualdad: dos artículos son iguales si tienen el mismo título, categoría y páginas.

Ejercicio 2: tipo Revista (2 ptos)

Implemente en el paquete `fp.asociacion` el tipo Revista utilizando **una clase**, de acuerdo con la siguiente descripción. **En la implementación de esta clase no se pueden utilizar streams**.

Propiedades:

- **Número**, de tipo String, consultable.
- **Fecha de publicacion**, de tipo LocalDate, consultable y modificable.
- **Ejemplares**, de tipo Integer, consultable y modificable.
- **Páginas fijas**, de tipo Integer, consultable.
- **Anuncios**, de tipo Integer, consultable.
- **Artículos**, de tipo List<Articulo>, consultable.
- **Número total de páginas**, de tipo Integer, consultable. Calculado como la suma de las páginas fijas y las páginas de todos los artículos.
- **Revista temática**, de tipo Boolean, consultable. Una revista se considera temática si tiene al menos un artículo largo.
- **Extension,** del tipo enumerado Extension, consultable. Puede tomar los valores BAJA, MEDIA y ALTA. Una revista es de extensión BAJA si el número total de páginas es menor de 32, MEDIA si está entre 32 y 64, ambas inclusive, y ALTA si es mayor de 64.

Constructores:

- C1: recibe un parámetro por cada propiedad básica del tipo.

Restricciones:

- R1: El número total de páginas de la revista debe ser múltiplo de 8.
- R2: Debe haber al menos un artículo en la revista.
- R3: La fecha de publicación de la revista debe ser un lunes.

Representación como cadena: una cadena con todas las propiedades básicas del tipo.
Criterio de igualdad: dos revistas son iguales si tienen el mismo número.
Criterio de ordenación: las revistas se ordenan por su número.

Ejercicio 3: Factoría (1 pto)

En la clase **FactoriaRevistas**, que se le da parcialmente implementada, implemente el método:

- *parseaRevista*: crea un objeto de tipo Revista a partir de una cadena de caracteres. La cadena de caracteres debe tener el mismo formato que las líneas del fichero CSV.

Ejercicio 4: Tratamientos secuenciales (6,5 ptos)

Dado el tipo **Asociacion** parcialmente implementado, complete su definición implementando todo aquello que considere necesario para cumplir con la descripción que se da a continuación. **En la implementación de esta clase no se pueden utilizar streams**. (0,25 ptos)

Propiedades:

- **Nombre**: nombre de la Asociación, de tipo String, consultable.
- **Anyo**: año en que se creó la Asociación, de tipo Integer, consultable.
- **Revistas**: lista de revistas publicadas por la Asociación, de tipo List<Revista>, consultable.

Constructores:

- C1: recibe un parámetro por cada propiedad básica del tipo.

Representación como cadena: una cadena con todas las propiedades básicas del tipo. La propiedad revistas se mostrará con el número total de elementos.
Criterio de igualdad: dos objetos de tipo Asociación son iguales si lo son su nombre y su año de creación.
Otras operaciones:

- *getNumerorevistas*: obtiene el número de revistas publicadas por la Asociación.
- *incorporaRevista*: dada una revista, la añade a la lista de revistas. Si ya está en ella, no hace nada.
- *obtieneFechaRevista*: dada una posición de la lista, obtiene la fecha en que se publicó la revista que se encuentra en esa posición. Si la posición no es válida, se elevará la excepcion `IllegalArgumentException`.

Se solicita implementar la clase **AsociacionUniversitaria** que extiende a la clase Asociacion, a la que añade una propiedad Universidad con el nombre de la universidad a la que pertenece la Asociación, y con la siguiente representación como cadena (0,25):
Representación como cadena: la misma que la clase Asociación, añadiendo al final el nombre de la Universidad entre paréntesis y todas las revistas separadas por un salto de línea.

Añada también a la clase **AsociacionUniversitaria** los siguientes tratamientos secuenciales. Todos los métodos deben ser implementados usando streams, excepto cuando se indique explícitamente lo contrario.

1. *getTotalPaginasPorCategoria:* dado un año, devuelve un Map que, a cada categoría de los artículos publicados en las revistas posteriores a ese año, le hace corresponder el número total de páginas de los artículos de esa categoría. Si el año es *null*, se tendrán en cuenta todas las revistas. **Implemente este método con bucles.** (1,5 ptos)
2. *getTitulosOrdenados*: dada una cadena, devuelve una lista con los títulos de todos los artículos que contengan en su título esa cadena, sin repetir ningún título. La lista estará ordenada por número de páginas de mayor a menor, y a igualdad de número de páginas por categoría en orden ascendente. Al buscar la cadena no se distinguirá entre mayúsculas y minúsculas. (1 punto)
3. *getMaxArticulosPorExtension*: dadas dos fechas, devuelve un SortedMap que, a cada extensión de las revistas cuya fecha de publicación sea igual o posterior a la primera y anterior a la segunda, le hace corresponder el número máximo de artículos de las revistas que tienen esa extensión. Las claves del SortedMap estarán ordenadas ascendentemente (1 punto)
4. *getPorcentajePaginasPorCategoria*: devuelve un Map que, a cada categoría de los artículos, le hace corresponder el porcentaje de páginas de los artículos de esa categoría respecto al total de páginas de los artículos de todas las categorías. Reutilice el método *getTotalPaginasPorCategoria*. (1 punto)
5. *getNumerosArticulosLargos*: devuelve una lista con los números de las revistas (ordenados por fecha de publicación) que tengan al menos dos artículos largos de categorías diferentes. (1,5 puntos)

Escriba una clase **TestAsociacion**. En la clase se leerán los datos del fichero y se probarán todos los tratamientos secuenciales, definiendo un método de test con los parámetros adecuados por cada tratamiento secuencial a probar. No se obtendrá la puntuación máxima del ejercicio si no se realiza el test y este ejecuta.

Los resultados esperados para el dataset proporcionado, con los valores indicados en los tests, son:

```
Factoría    ============================================================
Número de revistas publicadas por la Asociación: 100

Ejercicio 1 ============================================================
A partir del año 2024, el número total de páginas por categoría es:
{historia de la informática=44, desarrollo web=20, ingeniería de software=22,
infraestructura=12, inteligencia artificial=14, diseño=24, criptografía=22,
seguridad informática=6, ética profesional=36, bases de datos=36}

A partir del año null, el número total de páginas por categoría es:
{historia de la informática=174, ingeniería de software=154, desarrollo web=142,
infraestructura=114, educación=114, diseño=164, inteligencia artificial=192,
criptografía=170, seguridad informática=128, ética profesional=158,
comunidad=78, bases de datos=182}

Ejercicio 2 ============================================================
Títulos ordenados de artículos que contienen la cadena 'red':
[Redes neuronales, Auditorías de red]

Ejercicio 3 ============================================================
Entre las fechas 2020-01-01 y 2025-06-30, el máximo número de artículos por
extensión es:
{BAJA=1, MEDIA=3, ALTA=2}

Ejercicio 4 ============================================================
El porcentaje de páginas por categoría es:
{historia de la informática=9.830508474576272, ingeniería de
software=8.700564971751412, desarrollo web=8.022598870056497,
infraestructura=6.440677966101695, educación=6.440677966101695,
diseño=9.265536723163843, inteligencia artificial=10.847457627118644,
criptografía=9.6045197740113, seguridad informática=7.231638418079096, ética
profesional=8.926553672316384, comunidad=4.406779661016949, bases de
datos=10.282485875706215}

Ejercicio 5 ============================================================
Los números que tienen al menos dos artículos largos de categorías diferentes
son:
[200, 201, 206, 208, 209, 215, 216, 218, 220, 224, 234, 239, 240, 244, 250, 253,
264, 267, 269, 278, 281, 283, 287, 288, 292, 294]
```